

FIT2099 Assignment 2 Design Document [REQ 1-5]

MA_AppliedSession07_B

Erwyna Soo Wen Xin, 36555789

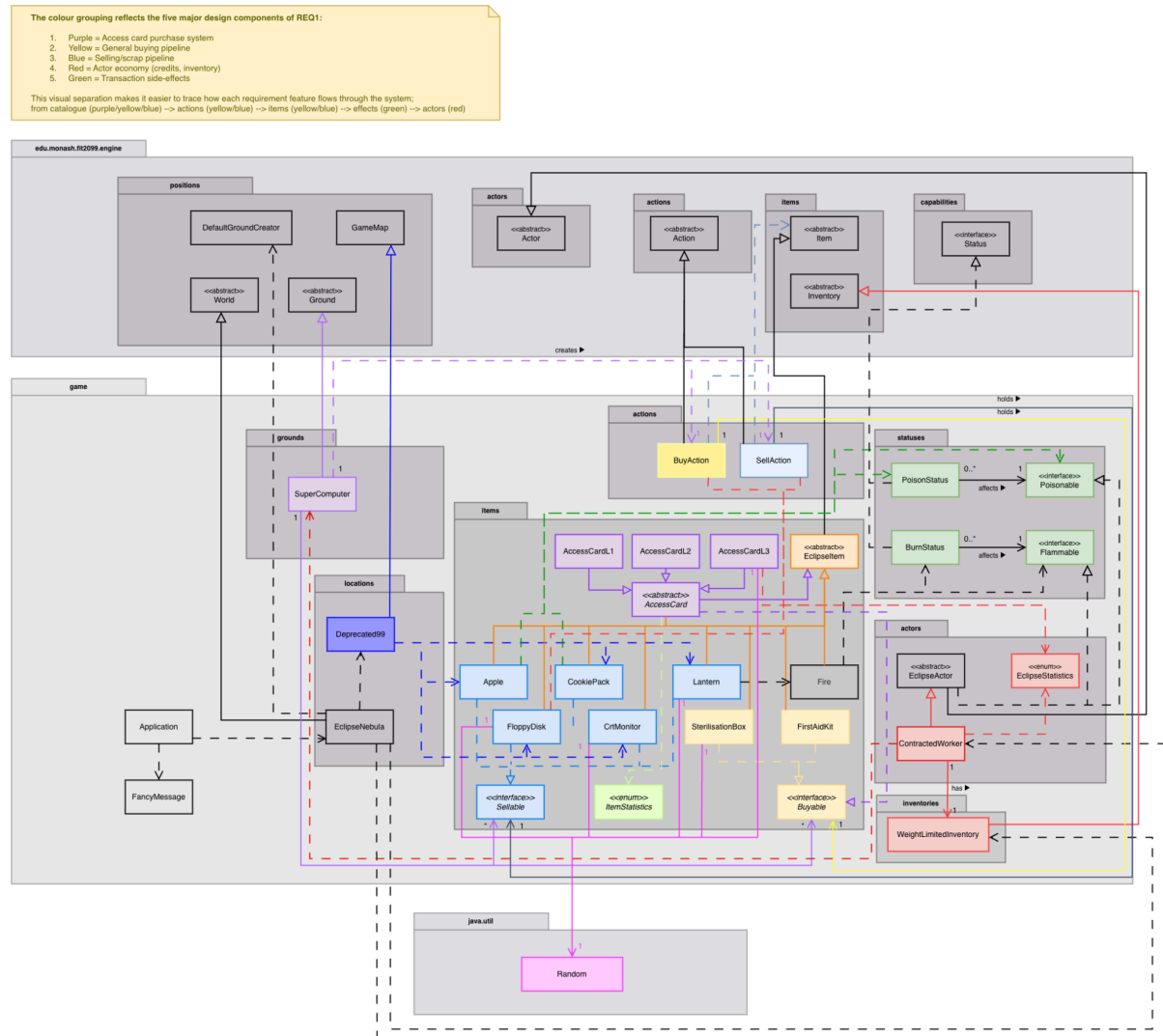
Yang Li Shen, 36031542

Edwin Chuah Wei-Hann, 35981229

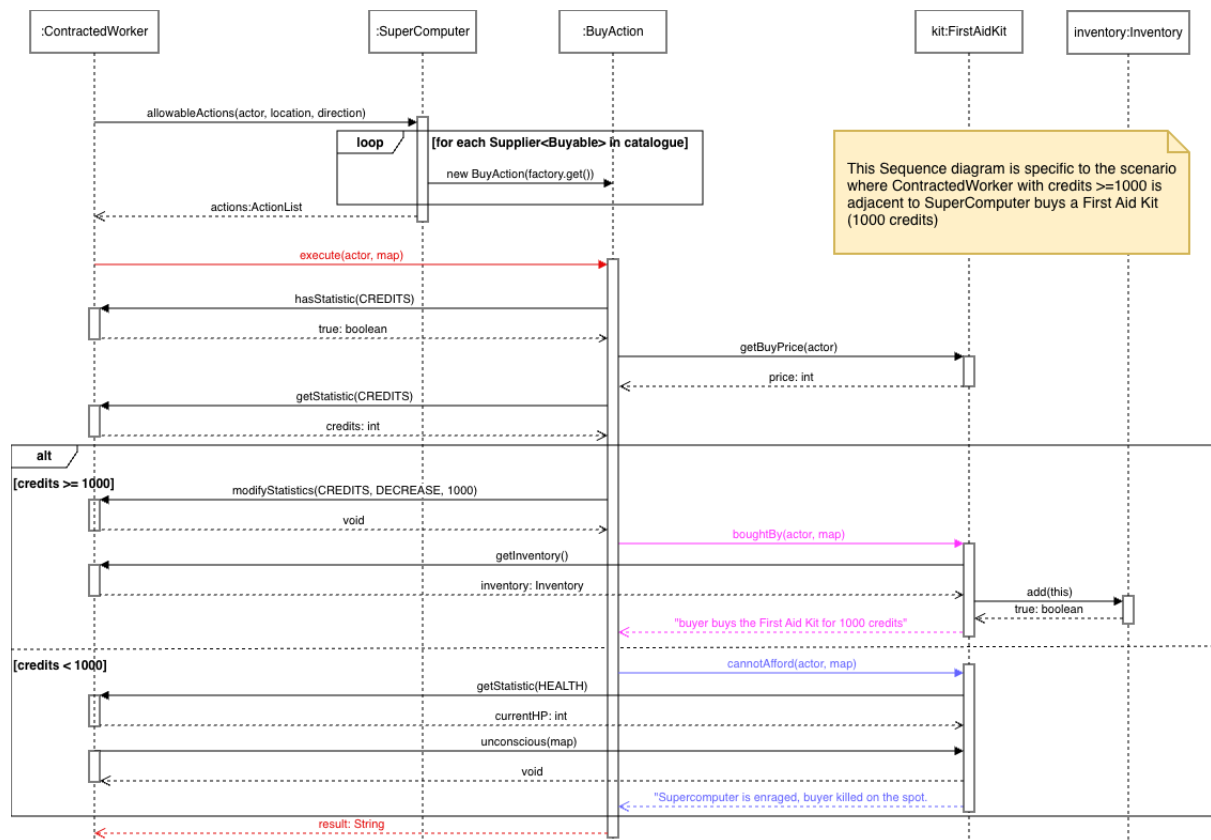
Lingsha Deng, 35894342

Edmond Cheong, 36367729

UML

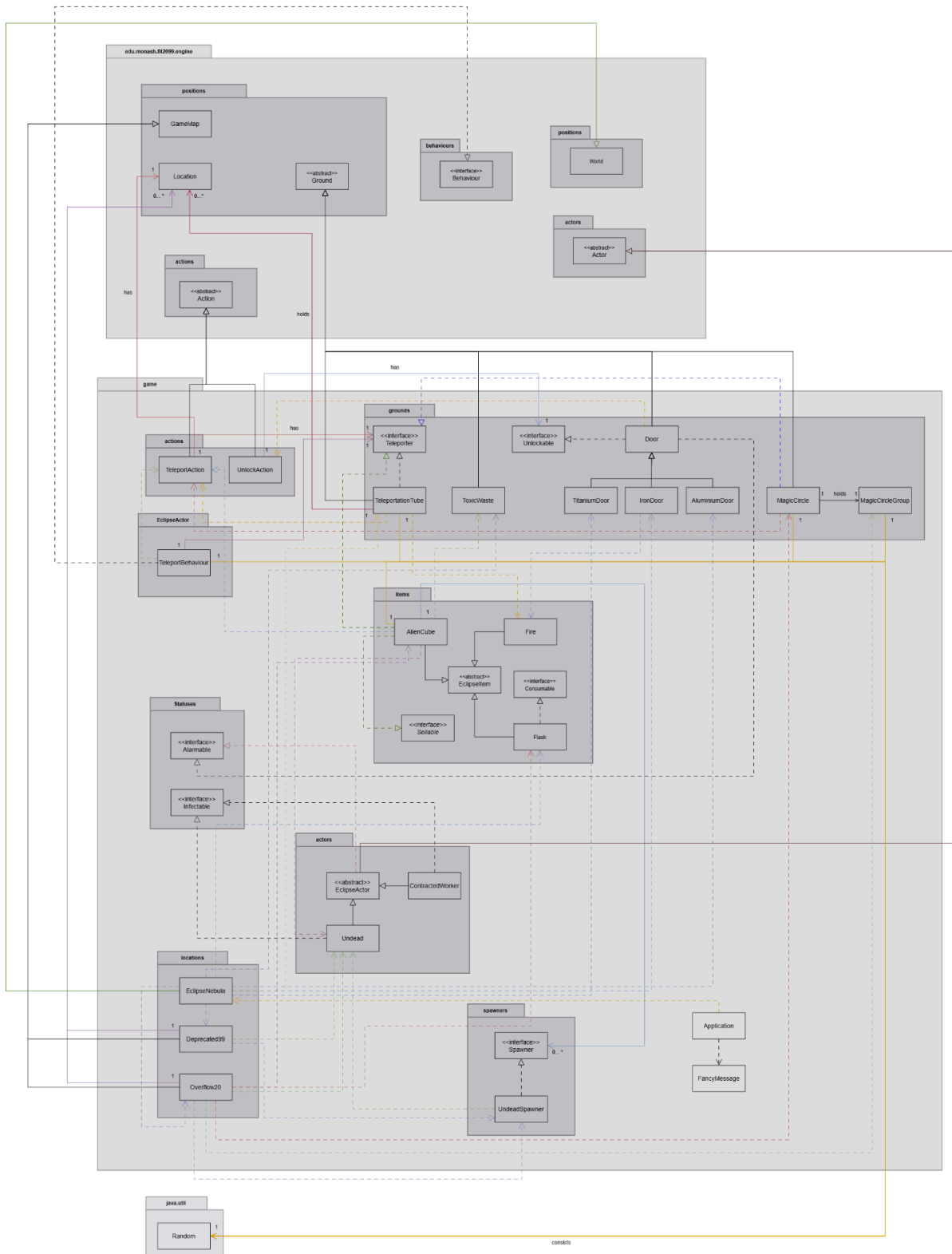


Sequence Diagram

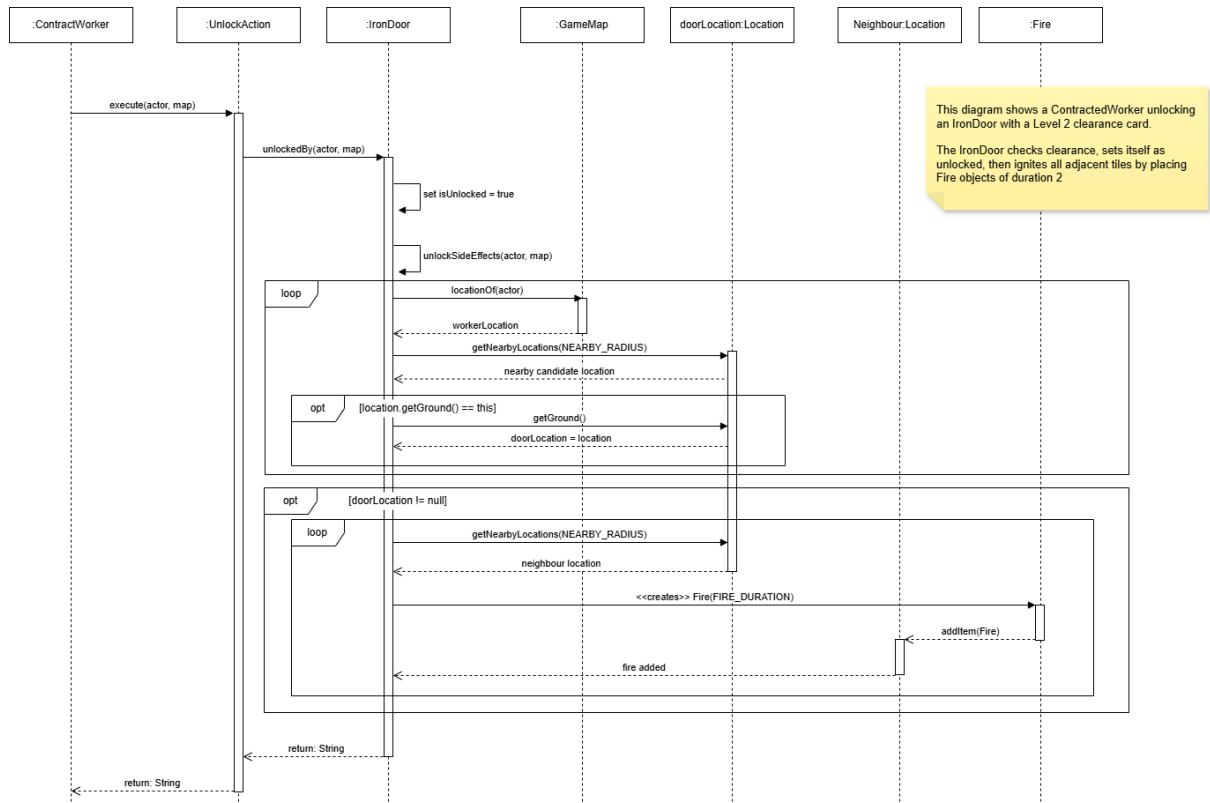


D. REQ 2 (Edmond Cheong, 36367729)

UML

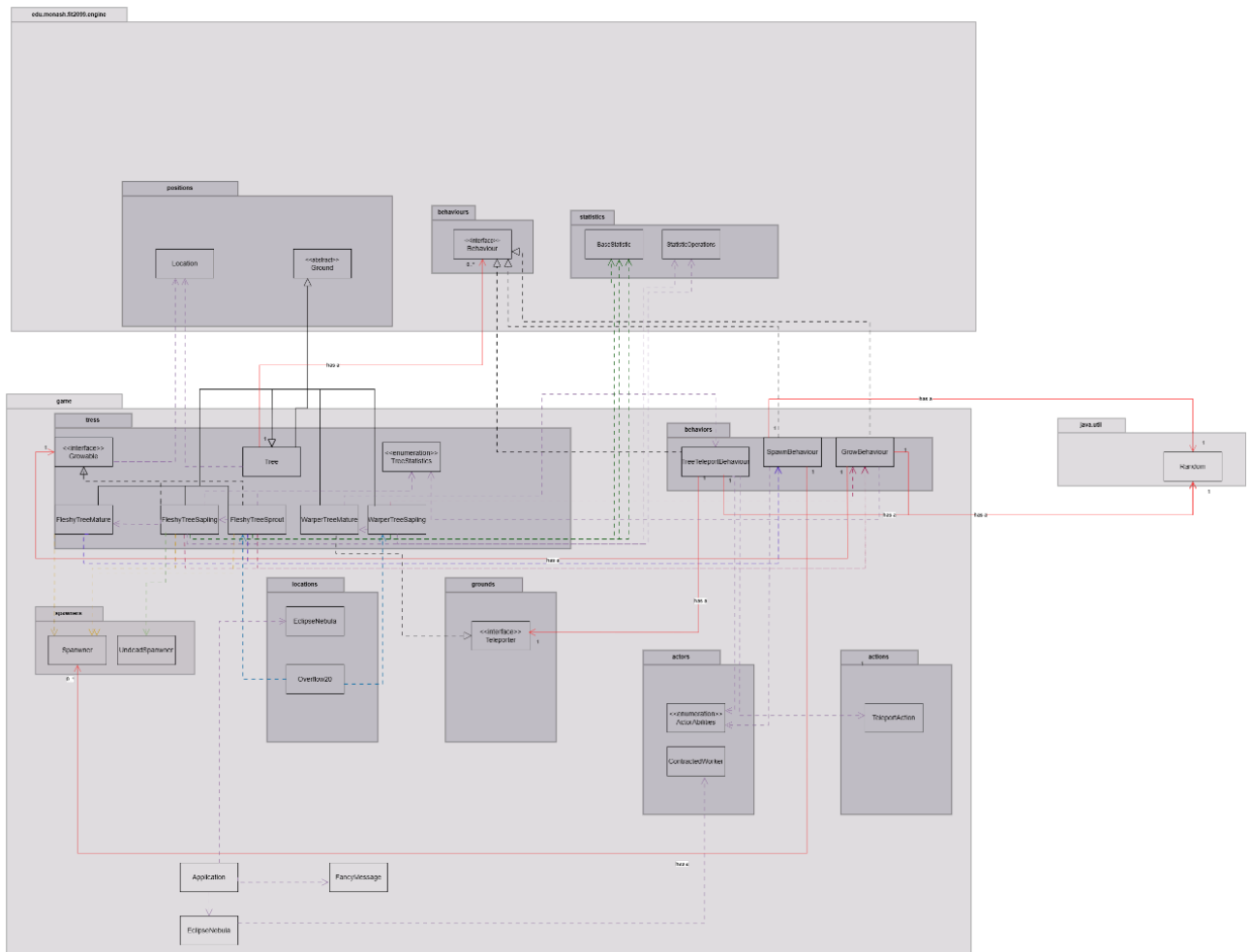


Sequence Diagram

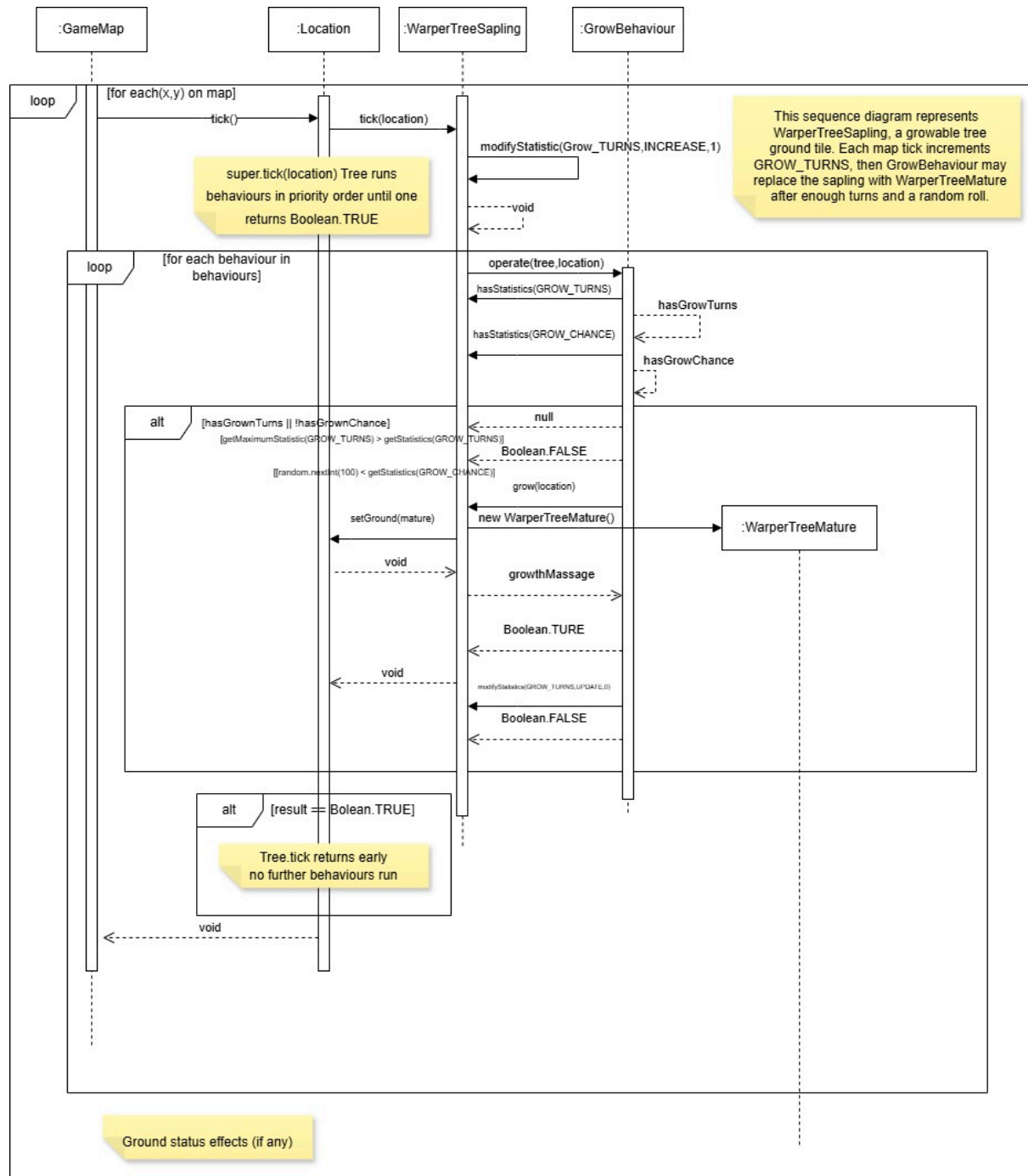


D. REQ 3 (Lingsha Deng, 35894342)

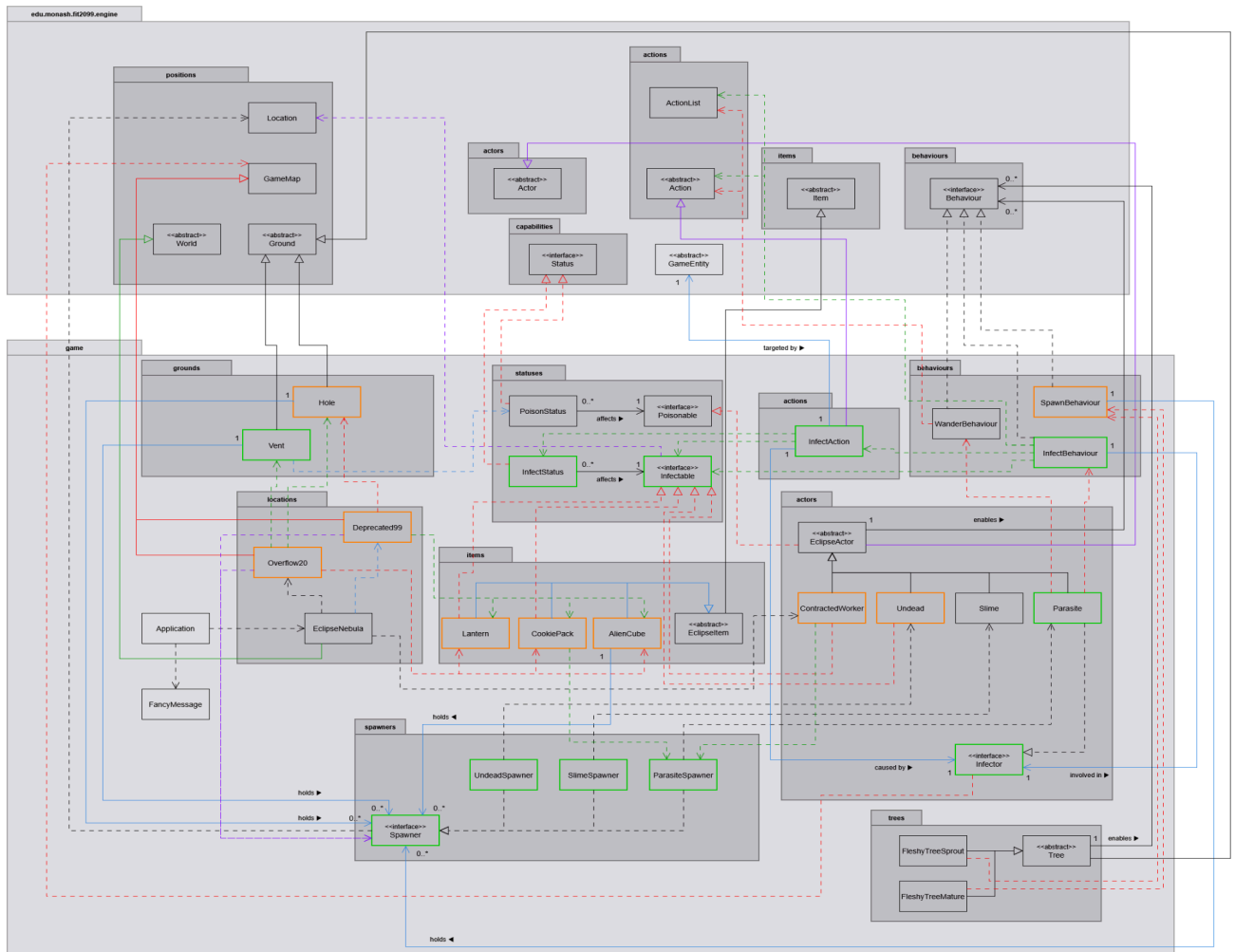
UML



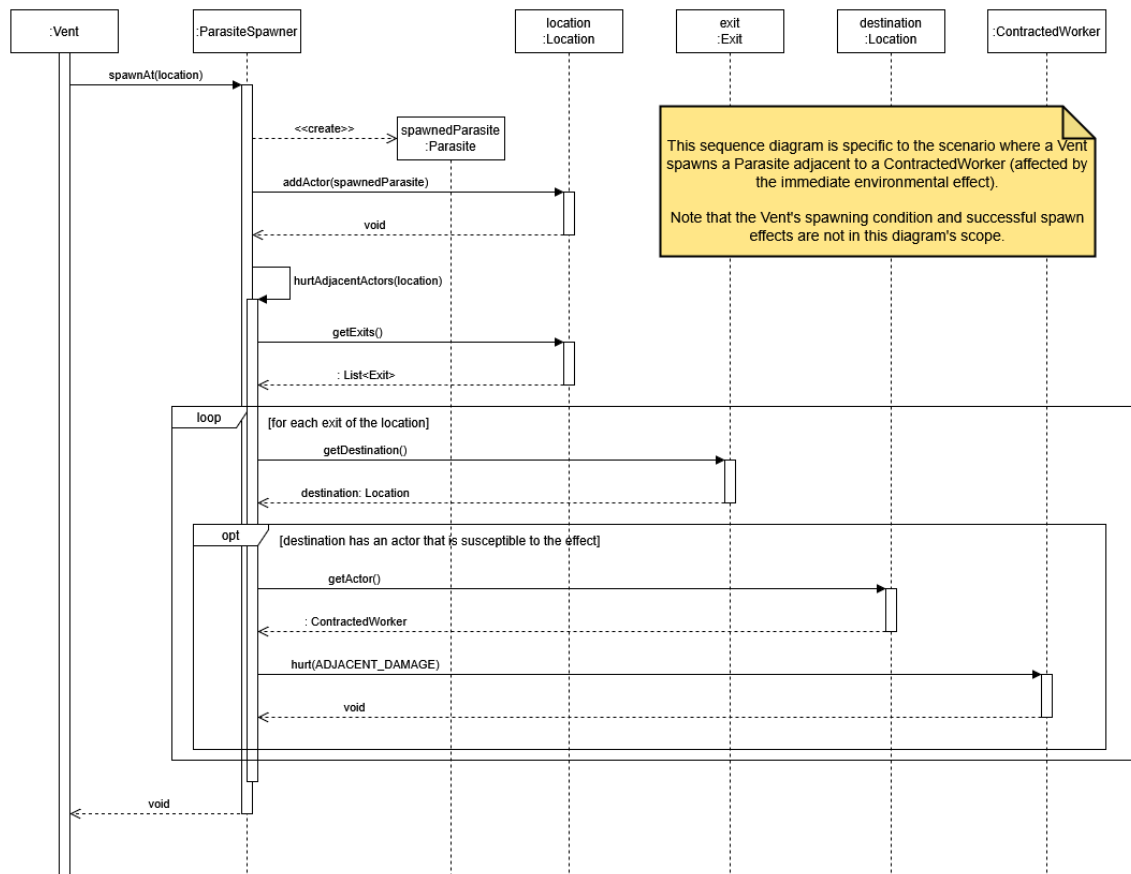
Sequence Diagram



Notes:
 Overlapping lines are given different colours for clarity.
 Classes may have different colours on their outline:
 Black classes are already existing classes (either from A1 or a previous REQ) that have not been changed for this requirement.
 Orange classes are existing classes that have been changed for this requirement.
 Green classes are new classes for this requirement.

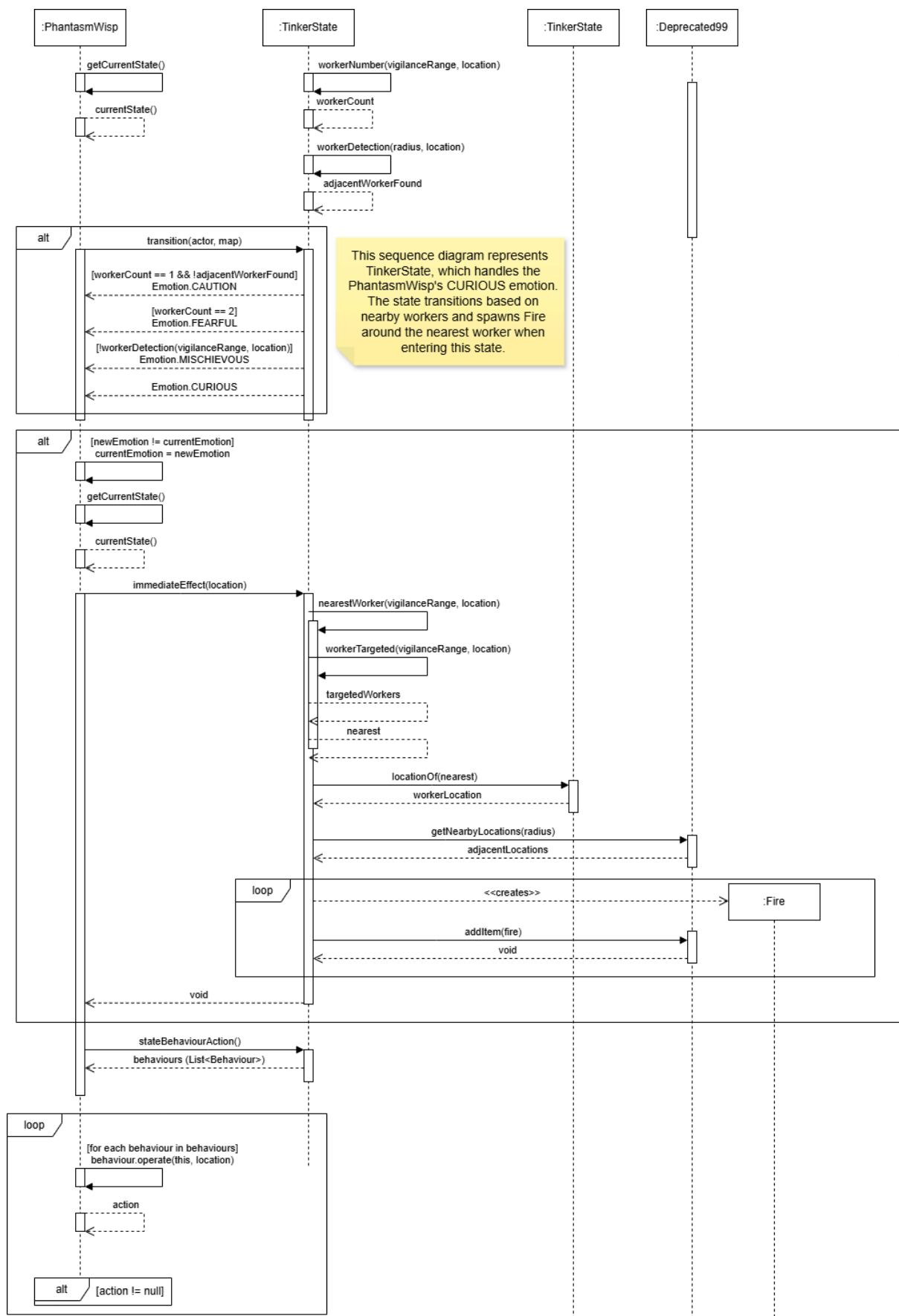


Sequence Diagram





Sequence Diagram



REQ 1 (Erwyna Soo Wen Xin, 36555789)

A. SuperComputer Catalogue and the Buy/Sell Pipeline

Design 1: Catalogue Injected as a List of Suppliers (Implemented Design)

The SuperComputer holds its catalogue as a list of suppliers, each producing a fresh Buyable item on demand. The catalogue is typed against the Buyable interface, so the terminal never needs to know the concrete type of any item it stocks. This means new items can be added to the catalogue without touching the SuperComputer class at all.

Pros	Cons
<u>Decouples the terminal from concrete items</u> , so it depends only on the Buyable abstraction and can stock anything that implements it.	<u>Adds a small layer of indirection</u> , since each catalogue entry is a factory call rather than a direct field.
<u>Produces fresh instances on every purchase</u> , preventing accidental sharing of a single item reference between actors.	<u>Requires the static factory method to be kept in sync</u> whenever a new buyable item is introduced.
<u>Supports the Open-Closed Principle</u> because new items extend the catalogue without modifying SuperComputer.	<u>Uses generic supplier types</u> , which may be slightly less familiar to read than a plain item list.

Design 2: Hard-Coded Catalogue Inside the SuperComputer (Alternative Design)

In this design, the SuperComputer would internally create its own item instances inside allowableActions(), constructing each item directly with the new keyword. The catalogue would be a fixed sequence of statements that name every item class explicitly.

Pros	Cons
<u>Straightforward to read</u> because the items the terminal sells are listed inline in one place.	<u>Tightly couples the terminal to every item class</u> , so adding or removing an item forces a change to SuperComputer.java.
<u>Requires no extra types</u> such as Supplier or generic parameters.	<u>Violates the Open-Closed Principle</u> because the terminal must be reopened and edited for every new buyable.
<u>Slightly fewer files to navigate</u> during initial implementation.	<u>Makes testing harder</u> , because the catalogue cannot be swapped out for a smaller test list.

Design 1 is used because it decouples the catalogue from the terminal through dependency injection. Adding a new purchasable item only requires extending the static factory method, without modifying SuperComputer's core logic (Open-Closed Principle). In contrast, Design 2 would tightly couple the terminal to every item class, forcing edits whenever a new item is introduced and reducing the ability to test the terminal independently.

B. Buying and Selling Through Capability Interfaces

Design 1: Buyable and Sellable Capabilities with Generic Actions (Implemented Design)

The ability to be purchased or sold is treated as a capability. The Buyable interface provides the contract for purchasable items, while Sellable does the same for sellable items. FirstAidKit, SterilisationBox, and AccessCard implement Buyable; Apple, CookiePack, Lantern, FloppyDisk, and CRTMonitor implement Sellable.

BuyAction and SellAction depend on these interfaces rather than concrete classes. BuyAction handles the purchase flow, delegating item-specific effects to the item's boughtBy method, keeping the action layer generic. Item-specific behaviour lives entirely inside each item, so actions remain free of type checks.

PROS & CONS of Design 1

Pros	Cons
<u>Eliminates duplicated action classes.</u> since one BuyAction handles every purchasable item and one SellAction handles every sellable item.	<u>Requires every item to implement the correct interface,</u> so introducing a new transactional item is slightly more deliberate.
<u>Encapsulates side-effects inside the item,</u> keeping the action layer thin and free of type checks.	<u>Pushes domain logic into items,</u> which means an item class can grow if its purchase behaviour is complex.
<u>Supports the Dependency Inversion Principle</u> because BuyAction and SellAction depend on abstractions rather than concrete items.	<u>Requires reading both the action class and the item class</u> to trace a complete transaction, which adds one extra file to navigate

Design 2: One Action Subclass Per Item (Alternative Design)

Every purchasable item would have its own dedicated action class (e.g., BuyFirstAidKitAction, BuySterilisationBoxAction), with item-specific logic implemented directly inside execute().

Pros	Cons
<u>Each transaction's logic</u> is visible in a single class without needing to follow a delegation chain.	<u>Causes class explosion</u> , with one new action subclass required for every new item.
<u>Items can stay as plain data</u> without implementing any extra interfaces.	<u>Duplicates the same credit-deducting and inventory-adding code</u> across every action subclass.
<u>No generic types or interface plumbing</u> are needed.	<u>Couples actions to concrete item classes</u> , violating the Open-Closed Principle and making the action layer fragile.

Design 1 is used because it eliminates duplicated action classes, with one BuyAction handling every purchasable item and one SellAction handling every sellable item. Unlike Design 2, this avoids repeating the same transaction logic across many action subclasses (DRY). Actions depend on abstractions rather than concrete item classes, which reduces the coupling between the action layer and individual items (Dependency Inversion Principle).

C. Item Side-Effects on Sale and Purchase

Design 1: Reusing the Engine Status System Through Capability Interfaces (Implemented Design)

Item side-effects (poison, burn, fire) are modelled as Status objects that integrate with the engine's per-turn lifecycle, so effects expire automatically without custom tracking. Actors declare supported effects through capability interfaces (Poisonable, Flammable), allowing items to apply effects without type checks. Environmental effects like Fire are treated as ordinary items, keeping the model consistent.

Note that item prices and side-effect values have been declared as named constants within each item class, eliminating connascence of meaning and changing it into connascence of name (weaker). This improves maintainability; adjusting effects requires modifying a single constant rather than hunting through multiple method calls.

PROS & CONS of Design 1

Pros	Cons
<u>Reuses the engine's tick lifecycle</u> , so effects expire automatically without any per-turn bookkeeping inside the actor.	<u>Adds extra interfaces (Poisonable, Flammable)</u> , which the actor must implement to be a valid target.
<u>Decouples items from concrete actor types</u> by routing every effect through a capability lookup rather than a class check.	<u>Requires an effect-application path</u> for each new status, since the actor must declare the matching capability.
<u>Keeps environmental effects (fire on adjacent tiles)</u> consistent with normal item placement, avoiding a separate effects subsystem.	<u>Side-effects are spread across several small classes</u> rather than living in one place.

Design 2: Custom Per-Effect Classes Managed by the Actor (Alternative Design)

Each effect would be its own dedicated class (e.g., PoisonEffect, BurnEffect) held in a collection on the actor. The actor would iterate over this collection each turn, applying damage and reducing duration manually.

Pros	Cons
<u>Behaviour for each effect</u> lives in one self-contained class.	<u>Forces the actor to manage effect lifecycles manually</u> , duplicating logic the engine already provides through Status.
<u>Avoids the engine's status system</u> , which may feel more lightweight at first.	<u>Bypasses an engine feature</u> that exists precisely for this purpose, reducing consistency with the rest of the codebase.
<u>Effects are easy to find</u> because they all sit in the actor's effect list.	<u>Makes the actor responsible</u> for every effect type, so adding new effects forces the actor class to grow.

Design 1 is used because it reuses the engine's existing Status lifecycle, avoiding the need to reimplement per-turn tracking and cleanup in the actor (DRY). It also decouples items from concrete actor types by routing effects through capability interfaces, which makes adding new effects more straightforward. Design 2 would require the actor to manually manage every effect's duration and damage, duplicating logic the engine already handles correctly.

D. Access Card Hierarchy

Design 1: Abstract AccessCard with Tier-Specific Subclasses (Implemented Design)

AccessCard is modelled as an abstract class that implements Buyable, with three concrete subclasses (AccessCardL1, AccessCardL2, AccessCardL3). Each subclass inherits shared state (price, weight, clearance level) from the parent and overrides boughtBy to define tier-specific purchase effects. AccessCardL2 deducts health from the buyer as a blood-sample fee, and AccessCardL3 has a 50% chance to charge a hidden fee on top of the purchase price. AccessCardL1 has no side-effect and simply enters the inventory. The clearance level is exposed through a single getter, allowing REQ2 doors to check a card's tier without inspecting its runtime class.

PROS & CONS of Design 1

Pros	Cons
<u>Shares common state and behaviour</u> (price, weight, symbol, clearance statistic) through inheritance, avoiding duplication across the three card classes.	<u>Introduces a small class hierarchy</u> for what is conceptually a single concept (a card).
<u>Encapsulates each tier's quirks</u> inside its own subclass, keeping behaviour local and easy to find.	<u>Requires a new subclass</u> every time a new clearance tier is introduced.
<u>Exposes clearance level through a single <code>getClearanceLevel()</code> getter</u> backed by an <code>ItemStatistics</code> value, so other systems can read a card's tier without inspecting its runtime type.	<u>Clearance level is stored as an integer statistic</u> rather than a dedicated enum, which is slightly less type-safe but integrates cleanly with the engine's statistic system.

The inheritance hierarchy introduces three classes instead of one, but each subclass isolates tier-specific behaviour (L2's blood sample, L3's hidden fee). This avoids conditional branching inside a single `AccessCard` class, which would grow messier as new tiers are added. Each tier's logic is easier to test independently, and adding a new tier only requires creating a new subclass without modifying `SuperComputer` or `BuyAction` (Open-Closed Principle, Single Responsibility Principle).

Design 2: Single AccessCard Class Parameterised by Level (Alternative Design)

In this design, there would be just one concrete AccessCard class, and the level would be passed into the constructor as an integer. All tier-specific behaviour, such as the L2 blood sample and the L3 hidden fee, would live inside a single boughtBy method that branches on the level value.

Pros	Cons
<u>Fewer classes to maintain</u> , with all card logic centralised in one file.	<u>Centralises behaviour</u> through level branching, which grows messier as new tiers are added.
<u>Easier initial setup</u> because there is no hierarchy to design.	<u>Violates the Open-Closed Principle</u> since a new tier forces a change inside the existing class rather than an extension.
<u>Constructor-driven</u> , so creating any tier is a single new statement.	<u>Mixes concerns inside one class</u> , weakening cohesion and making each tier's behaviour harder to locate.

Conclusion of REQ1

The implemented design separates the SuperComputer transaction into small, loosely-coupled components. The terminal presents the catalogue, actions orchestrate credit movement, items handle transaction-specific effects, and the engine's status system manages effect lifetimes; each component interacts through narrow interfaces rather than concrete classes (**Single Responsibility Principle**).

The design absorbs growth through extension rather than modification. New sellable or purchasable items can be added by implementing the relevant interface and extending the factory method, without touching existing actions or the terminal. Actions depend on abstractions (*Buyable*, *Sellable*) rather than concrete item classes, meaning high-level transaction logic is never affected by changes to individual items (**Dependency Inversion Principle**). Access card tiers follow the same pattern; new tiers extend the hierarchy without modifying the shared parent or any other part of the system (**Open-Closed Principle**). The trade-off is a larger number of small types, but this keeps each component focused and the system maintainable as the game grows across future requirements.

REQ2 (Edmond Cheong, 36367729)

A. Teleportation Device Abstraction

Design 1: Implement teleportation logic directly inside each teleporting class (TeleportationTube, MagicCircle, AlienCube) by duplicating the move actor call and arrival effects inline within their respective allowableActions() methods.

PROS	CONS
Simple to implement, no additional interface needed. Each class handles its own movement logic in one place.	Violates DRY the actor movement call and arrival effect logic would be duplicated across all three teleporting classes.
All teleport logic is visible directly inside the class that triggers it.	Violates OCP adding a new teleportation device requires independently reimplementing movement logic with no shared contract to enforce correctness.

Design 2: Add a Teleporter interface with a single teleport(Actor, GameMap, Location) method. All teleportation devices (TeleportationTube, MagicCircle, AlienCube) implement this interface and define their own arrival effects inside teleport(). A shared TeleportAction delegates to teleporter.teleport() in its execute() method.

PROS	CONS
Satisfies SRP TeleportAction is solely responsible for invoking teleportation; each device class is solely responsible for defining its own arrival effects.	Teleporter is currently placed in game.grounds, which is a mild coupling smell since AlienCube (an item) also implements it.
Satisfies OCP the new teleportation devices only require a new class implementing Teleporter with no changes to TeleportAction.	
Satisfies DRY single TeleportAction class is reused across all three devices with zero duplication of menu.	

Design 1 has connascence of algorithm, because movement and arrival effect logic would be duplicated across all three device classes with no shared contract.

On the other hand, Design 2 reduces this to connascence of name on teleport and connascence of type on Teleporter, both compiler enforced. A mild connascence of position remains in the teleport(Actor, GameMap, Location) signature as all implementors must agree on parameter order. Lastly, a connascence of identity persists as each device class declares its own static Random, making malfunction behaviour difficult to test deterministically.

B. Door Tier Hierarchy

Design 1: Implement AluminiumDoor, IronDoor, and TitaniumDoor as completely independent classes each duplicating lock state management, clearance checking, alarm handling, and allowableActions() logic internally.

PROS	CONS
Simple, no abstract class needed, each door is entirely self contained.	Violates DRY, lock state (isUnlocked), alarm state (isAlarmed), clearance checking, and allowableActions() logic would be duplicated across all three door classes.
	Violates OCP fixing a bug in unlock logic (e.g. alarm relocking) requires changing every door class individually.
	Violates LSP without a shared supertype, doors cannot be substituted for one another where a generic Door reference is expected.

Design 2: Implement an abstract Door class extending Ground and implementing Unlockable and Alarmable. It owns all shared state (isUnlocked, isAlarmed, securityLevel) and behaviour (clearance checking, allowableActions(), alarm toggling). Subclasses only override the abstract unlockSideEffects(Actor, GameMap) method to define tier specific hazards or benefits.

PROS	CONS
Satisfies DRY lock state, clearance logic, and alarm handling are written once in Door and inherited by all three subclasses.	Extending Ground introduces tight coupling to the engine's ground hierarchy changes to Ground may require updating Door and all subclasses.
Satisfies OCP adding a new door tier requires only a new subclass overriding unlockSideEffects() with no changes to existing classes.	Subclasses must remember that unlockedBy() in Door already sets isUnlocked = true before calling unlockSideEffects(), requiring awareness of the parent's call order.
Satisfies LSP all three door types are interchangeable as Door references, and the Alarmable/Unlockable contracts are upheld uniformly.	

In Design 1 introduces connascence of algorithm. Lock state, clearance checking, and alarm logic would be independently reimplemented in each door class.

As compared to Design 2 which reduces this to connascence of name on unlockSideEffects and connascence of meaning on the security level integers (1, 2, 3), which all door classes and canActorUnlock() agree on implicitly with no single authoritative source. A deliberate connascence of execution exists in the Template Method: Door.unlockedBy() sets isUnlocked = true before calling unlockSideEffects(), so subclasses must be aware the door state is already mutated when they receive control.

C. Toxic Waste as Permanent Ground

Design 1: Implement toxic waste damage by adding a poison like status to any actor that steps onto a corrupted tile, tracked inside the actor or via the existing PoisonStatus class.

PROS	CONS
Reuses the existing status structure from REQ1's poison system.	Misrepresents the mechanic toxic waste is a persistent environmental hazard tied to a location, not a temporary timed effect on an actor. A status expires; toxic waste never does.
	Requires the tile to reapply the status every turn, adding fragile reapplication logic.
	Does not accurately model the spec's intent that the ground itself is permanently corrupted.

Design 2: Implement ToxicWaste as a new Ground subclass that overrides tick(Location) to deal 1 damage per turn to any actor standing on it.

AlienCube.teleport() calls location.setGround(new ToxicWaste()) on all adjacent tiles at the source location.

PROS	CONS
Satisfies SRP ToxicWaste is solely responsible for its own perturn damage logic; AlienCube is solely responsible for triggering the corruption.	ToxicWaste permanently replaces whatever ground was there before, with no way to restore the original tile. This is intentional as the spec but is irreversible by design.
Accurately models the spec corrupted ground is a permanent map change, not an actor status, and Ground.tick() is the correct hook for location based perturn effects.	
Satisfies OCP future ground types with perturn effects follow the same tick() override pattern with no changes to existing classes.	

Design 1 introduces connascence of timing. The tile would need to reapply a poison status every turn, coupling the ground and actor status systems to execute in the correct order each turn.

In Design 2 produces only a single connascence of execution within AlienCube.teleport(): the source location must be captured before moveActor is called, otherwise the wrong tiles are corrupted. This is localised to three consecutive lines in one method. Beyond this, ToxicWaste.tick() has no dependency on any other class's state or timing.

D. Magic Circle Peer Registry

Design 1: Give each MagicCircle a direct List<MagicCircle> reference to all other circles on the map, populating it manually at construction time in Overflow20.

PROS	CONS
Its direct, each circle can immediately query its peers without an intermediary object.	Violates DRY every circle holds a full copy of the peer list, meaning adding or removing a circle requires updating every other circle's list individually.
	Tight coupling between MagicCircle instances circles are aware of each other's concrete type rather than just their locations.

Design 2: Introduce a MagicCircleGroup registry class. Each MagicCircle holds a reference to the shared group and calls group.randomPeerExcluding(location from) to get a destination. Overflow20 registers each circle's location into the group after placing it on the map.

PROS	CONS
Satisfies DRY peer selection logic is written once in MagicCircleGroup.randomPeerExcluding() and reused by every circle in the network.	Overflow20 must remember to call circles.register() after placing each circle forgetting this silently results in a circle with no valid peers and no teleport action offered, which can be hard to diagnose.
Satisfies SRP MagicCircle is responsible for offering the teleport action; MagicCircleGroup is solely responsible for managing the peer list and selecting a random destination.	
Satisfies OCP adding more circles to the network only requires placing them and registering their location; no changes to MagicCircle or MagicCircleGroup are needed.	

Design 1 has connascence of identity each circle holds direct references to every other circle instance, multiplying the change surface with every addition.

Design 2 reduces this to one contained connascence of identity, all circles must share the exact same MagicCircleGroup instance, localised to Overflow20.setMagicCircles(). A connascence of timing is introduced as the noted con: setGround must precede circles.register() for each circle, as randomPeerExcluding() filters by map reference equality and a location not yet placed on a map will fail silently.

Summary

The diagram above (REQ 2) shows the final design of REQ2, which applies Design 2 across all four decisions. The core principle driving every design choice in REQ2 is that each class owns exactly one responsibility, shared behaviour is written once and inherited or delegated rather than duplicated, and the system can be extended with new teleportation devices or door tiers without modifying any existing class.

The Teleporter interface establishes a shared contract across all three teleportation devices. Since each device has meaningfully different arrival effects, fire rings for TeleportationTube, flask spawning for MagicCircle, and toxic corruption for AlienCube; polymorphism through teleport() is more appropriate than a shared base class. TeleportAction depends only on the Teleporter abstraction, satisfying the Dependency Inversion Principle, and any future device implementing Teleporter is automatically compatible with the existing action infrastructure (Open/Closed Principle).

The abstract Door class eliminates the duplication that would arise from three independent door implementations. Lock state, alarm toggling, clearance verification, and allowableActions() are defined once and inherited. The Template Method pattern is applied through the abstract unlockSideEffects() hook, Door.unlockedBy() defines the invariant sequence (set unlocked, then apply side effects) while each subclass fills in only what is unique to its tier. This means AluminiumDoor, IronDoor, and TitaniumDoor each contain only lines that distinguish them from one another.

ToxicWaste is modelled as a permanent Ground subclass rather than a reused actor status because the corruption is a lasting map mutation, not a timed effect on an actor. Using Ground.tick() as the damage hook correctly couples the effect to the location rather than to whoever happens to be standing there, and accurately reflects the spec's intent that the ground itself is irrecoverably changed.

MagicCircleGroup decouples circle to circle awareness behind a registry. Each MagicCircle depends only on the group abstraction rather than holding direct references to peer instances, satisfying the Dependency Inversion Principle. Peer selection and same map filtering logic live entirely within MagicCircleGroup.randomPeerExcluding(), meaning the rule that teleportation is within map only is enforced in exactly one place (DRY).

REQ3 (Lingsha Deng, 35894342)

Design 1: Prioritized Behaviour Pipeline

Abstract Tree stores behaviours in a TreeMap keyed by priority. FleshyTreeSprout registers GrowBehaviour then SpawnBehaviour; WarperTreeMature registers TreeTeleportBehaviour. Growable stages increment GROW_TURNS before Tree.tick, which runs behaviours until one returns TRUE. One-action-per-turn rule follows because successful growth ends the tick before spawning. KISS uses one scheduler; DRY avoids duplicating turn control. SRP splits scheduling from effects; OCP adds flora via new behaviours. Example: on turn 20 a nearby worker triggers spawn only if the 25% growth roll fails. Maintenance stays stable because warp and spawn rules do not interleave in one method. Connascence of position (priority keys) and execution (first TRUE stops the chain) are moderate: coupling is localized to registration order. Extension: ThornSnareBehaviour depends on Tree and registers on a new ThornSprout.

Pros	Cons
Single loop enforces one flora action per turn.	Priority values are fragile implicit contracts.
Effects compose without editing Tree.	Return-value conventions must stay uniform.
Warp, grow, and spawn stay in separate units.	Turn outcomes need tracing across behaviours.

Design 1: Alternative Design

Each stage would override tick with inline growth, spawn, and warp checks, returning after the first success. Growth timers and adjacency scans repeat across classes, weakening DRY. KISS looks simpler locally; SRP suffers when one method owns randomness, actors, and map updates. Example: sprout logic would duplicate worker detection now in SpawnBehaviour. Connascence of algorithm is stronger than the pipeline because parallel copies must stay aligned. Trade-off: fewer types, more regression risk when tuning priorities. Extension: ParasiticTree adds another large tick instead of a behaviour.

Pros	Cons
All logic visible in one method per stage.	Single-action ordering duplicated per class.
No behaviour map to document.	Spawn and grow code diverge over time.
	Harder to reuse warp logic on warper mature.

Harder to reuse warp logic on warper mature.

Design 2: Growable Ground Stages

Growable.grow replaces the tile through location.setGround, giving each stage its own class (FleshyTreeSprout, FleshyTreeSapling, WarperTreeSapling, etc.). GrowBehaviour reads TreeStatistics for 20/25-turn windows and 25%/50% chances, then delegates replacement. Mature fleshy trees appear only when sapling growth constructs FleshyTreeMature with spawners. Encapsulation keeps glyphs and timers per stage. DRY shares timed growth; ISP keeps non-growing mature trees off Growable. Example: after 25 sapling turns, a failed 50% roll resets the counter; success becomes Y. Scalability adds species by chaining new grounds. Connascence of type (GrowBehaviour needs Growable) and meaning (enum stats encode growth rules) are moderate. Extension: SporeBud implements Growable and yields SporeCanopy, reusing GrowBehaviour.

Pros	Cons
Map display always matches the active stage class.	Each stage needs a new class and grow link.
One growth algorithm serves all evolving flora.	Stat names must match GrowBehaviour expectations.
Stage-specific stats stay encapsulated.	Class count grows with every species line.

Design 2: Alternative Design

One FleshyTree would use a Stage enum, mutating character, timers, and spawn flags internally. Fewer files aid KISS superficially, but warper and fleshy rules converge in one class, hurting SRP and OCP. Example: maturity would flip enum rather than call setGround. Connascence of meaning intensifies inside the enum-driven class, weaker between files but harder to change one species. Trade-off: less allocation, denser conditionals. Extension: WARPED adds more switches, increasing coupling.

Pros	Cons
Fewer types for the fleshy chain.	Fleshy spawn and warper warp share one switch tree.
Transitions avoid new ground instances.	Enum edits risk breaking unrelated stages.
Shared fields can persist across stages.	Harder to read stage behaviour from the type alone.

Design 3: Injected Spawner Strategy

SpawnBehaviour takes List<Spawner>; Overflow20 passes SlimeSpawner to sprouts, while FleshyTreeSapling.grow attaches UndeadSpawner to mature trees. spawnAt creates actors and runs slime drop or undead HP reactions. DIP keeps trees on Spawner; OCP adds creatures via new spawner classes. Example: worker beside Y picks a neighbour tile and UndeadSpawner.spawnAt runs. Connascence of name (TREE_ACTIVATOR) and type (list element contract) stay mild. Extension: FleshyTreeMature(List.of(new SlimeSpawner(), new UndeadSpawner())) mixes hazards without changing SpawnBehaviour.

Pros	Cons
Spawn consequences live with the creature type.	Misconfigured lists fail at runtime.
Maps configure pools per tree instance.	Worker detection remains in SpawnBehaviour.
Trees avoid duplicating undead or slime effects.	Random pool choice hides outcomes until spawn.

Design 3: Alternative Design

Trees would construct Slime or Undead directly when workers are adjacent. Undead boosting and item drops duplicate wherever spawning occurs, breaking DRY. Connascence of type to concrete actors is stronger than the interface approach. Trade-off: linear reading, wide change blast radius. Extension: each tree edits its own spawn block for parasites.

Pros	Cons
Direct spawn calls are easy to follow.	Environmental reactions repeat across callers.
No spawner list wiring in maps.	Trees couple to actor constructors.
Minimal abstraction for two creatures.	Mixed pools need new conditionals per tree.

Comparison of Alternative Designs

Fat tick overrides maximize algorithm connascence across stages. Enum staging reduces inter-class coupling but concentrates meaning connascence in one growing class. Direct spawning minimizes interfaces yet binds trees to concrete actors. The enum alternative weakest spreads coupling across files, but its class becomes the main maintenance hotspot.

CONCLUSION

The diagram above shows the final design of REQ 3, which utilizes Design 1's behaviour pipeline with Design 2's growable ground stages and Design 3's injected spawners, while WarperTreeMature implements Teleporter for root warps. Design 1 was chosen because a single stop-on-success rule implements single-threaded flora turns reliably. Design 2 mirrors distinct stages and timers without mixing species in one object. Design 3 isolates spawn side effects shared with other map features. KISS favors small collaborators over monolithic ticks; DRY shares growth and spawn orchestration; SOLID separates scheduling, lifecycle, and spawning. Connascence of execution concentrates in Tree.tick and priorities, which is easier to adjust than scattered algorithms. Trade-offs include more classes and careful priority wiring; fat-tick, enum, or direct-spawn alternatives may suit larger worlds driven by data files.

REQ4 (Edwin Chuah Wei-Hann, 35981229)

Decision 1a: Multiple targets need to be able to be infected by the parasite, each with different effects.

Chosen design: Create an interface Infectable and let each concrete class that can be infected implement them. Each of those classes themselves will define what effects they have when infected in the infection method, which is called by InfectStatus each turn.

Advantages	Disadvantages
If more entities are added that can be infected, they just need to implement Infectable and provide the logic for how they react to the infection. No need to modify any other classes.	Adding an interface would make the code more complex.
Each concrete class implementing Infectable have very different effects to each other (avoids impacting readability by keeping them separate).	-

Alternative design: No Infectable interface. Instead, in InfectStatus, there will be a switch statement. Every turn will take the infected target (GameEntity), use instanceof and downcasting to check if the target is a certain concrete class, then define those effects for each case.

Advantages	Disadvantages
Code seems simpler without the interface, and every effect related to the infection is kept within one class.	Adding a new entity that can be infected will require modifying InfectStatus to add another case to the switch statement to check for that new entity.

The chosen design was used as it is more extensible. When a new class implementing Infectable is added, the only thing that needs to be done is to define how to handle the infection logic in that class. This does not require modification of any other classes, which adheres to the Open-Closed Principle (OCP). Furthermore, this adheres to the Dependency-Inversion Principle (DIP) since, unlike the alternative design, InfectStatus has a dependency on the Infectable interface rather than each of the concrete classes (ContractedWorker, Undead etc.). This removes the tight coupling between InfectStatus and those concrete classes.

The chosen design here is actually similar to the design around poison and burn effects in Assignment 1, with the same benefits as well.

Decision 1b: The parasite needs to be able to infect adjacent entities.

Chosen design: Create InfectBehaviour and InfectAction. InfectBehaviour will be used by the parasite. InfectBehaviour returns an InfectAction for the parasite to execute.

Advantages	Disadvantages
InfectBehaviour standardizes the logic for an actor infecting other entities. New EclipseActor classes could just have this behaviour added to their behaviours rather than needing to repeat code.	Multiple additional classes are made to handle infecting, increasing complexity of the design.

Alternative design: In the Parasite's playTurn method, before iterating through any of its behaviours, perform the logic related to infecting (looking for an Infectable entity in adjacent locations, giving them InfectStatus and killing the parasite itself) directly.

Advantages	Disadvantages
Avoids adding additional classes, keeping the design simpler.	Any future actors that can infect other entities cannot reuse this. They would have to define the same logic in their playTurn method, which is code duplication.

The chosen design was used as the logic for infecting is defined in one place, InfectBehaviour. Since it might be possible in the future that a new EclipseActor gets added that has the ability to infect others as well, it does not need to define the same logic again, instead just using InfectBehaviour which already exists. This adheres to DRY as there's no code duplication necessary, and promotes code reusability.

Also, unlike the alternative design, instead of having Parasite be responsible for finding Infectables, giving the status effect and dying instantly, this design moves the responsibility of inflicting the status effect to InfectAction* and finding Infectables to

InfectBehaviour, demonstrating separation of concerns and adhering to the Single Responsibility Principle (SRP).

*The parasite dying instantly will be caused by InfectAction, but this is part of another design decision.

Note that the behaviour priorities for Parasite (and other EclipseActors) have been declared as class constants (static final), eliminating the magic numbers that were present from the design in Assignment 1. This changes the connascence of meaning into connascence of name (weaker), improving readability and maintainability.

Decision 1c: The parasite needs to instantly die when it successfully infects a target.

The chosen design uses an Infector interface, which requires concrete classes that implement it define the effect of what happens to it when it infects a target. Parasite implements this and the effect is to just die. InfectAction will give the InfectStatus to the target, and then activate this effect to the Infector.

An alternative design would be to have InfectAction kill the Parasite directly, which doesn't need an interface and is simpler to implement.

But consider, hypothetically, a new EclipseActor that can infect others, but doesn't die from infecting (some other effect). The current design is more flexible and extensible as it easily supports this addition without needing to change InfectAction to handle the new effect for that new class. This adheres to the OCP.

Decision 2a: The hole in Deprecated99 and Overflow20 needs to spawn different creatures.

Chosen design: Holes will have an attribute which is a list of what it can spawn (Spawners*). This list is defined and passed in by each GameMap when creating Holes (as part of its map structure).

*This is part of another decision discussed later.

Advantages	Disadvantages
Adding new GameMaps does not require modifying Hole since GameMap defines what their Holes can spawn.	DefaultGroundConstructor no longer works for Holes. GameMaps need to set their Hole grounds manually, which is less convenient.

Alternative design: When spawning, Holes will check which GameMap they're on (this is possible since its tick method provides its location, where it can get its GameMap). A switch statement, instanceof and downcasting is then used to determine the creatures it can spawn.

Advantages	Disadvantages
Still allows the DefaultGroundConstructor to be used for Holes, allowing for easier set up of the GameMap's structure.	Adding new GameMaps require also modifying each Hole, or even other ground spawners like Vents if designed similarly to Hole.

The chosen design was used as it is more extensible. Adding more GameMaps requires no modification to the Hole class. This adheres to the OCP. While the alternative is more convenient to set up new GameMaps, the disadvantages of violating OCP and having less readability and maintainability in Hole outweighs it. Additionally, this design uses dependency injection for the Hole's list of Spawners, which means Hole has a dependency to the Spawner interface rather than to each concrete class implementing Spawner. This demonstrates the DIP.

Decision 2b: Hole and Vent have effects after successfully spawning a creature. Should an interface (e.g. GroundSpawner) be added where this effect is to be defined?

The chosen design doesn't have this interface, simply because it's redundant. These effects are called internally within each respective class and is not used anywhere else. Adding this interface unnecessarily increases complexity and would be a violation of KISS.

Decision 3: Creatures spawned trigger an immediate environmental effect, no matter what spawned them.

Chosen design: Create a Spawner interface and concrete classes (e.g. UndeadSpawner) that implements this. These will handle spawning an actor when given a location, as well as any other behaviours (like those immediate effects).

Advantages	Disadvantages
Any creature spawned using the Spawner will always trigger the immediate effect (if there's one defined for the respective creature).	Additional interface and concrete classes add complexity to the design.
Avoids code duplication on the entity that spawns the creature (no need for each one to check if the creature has an effect).	-

Alternative design:

(Note that this alternative doesn't have a Spawner interface, so Hole/Vent would have a `List<Supplier<Actor>>` instead of `List<Spawner>`.)

Add an `EnvironmentalTriggerer` interface for Undead, Slime and Parasite to implement. Those classes will define the environmental effect. Whenever creatures are spawned, the class spawning them (e.g. Hole) needs to check if the creature implements this interface (via downcasting), and activate the effect if it does.

Advantages	Disadvantages
Does not need an additional interface and additional concrete class for each creature that has the immediate effect.	Everything that spawns creatures must check if the creature implements the interface, then activate the effect. Code duplicating aside, this might be easily missed, leading to logic errors.
-	Involves downcasting, although this uses the engine's code which isn't too much of an issue.

The chosen design was used as each Spawner concrete class guarantees that when it spawns a creature, it will trigger the immediate effect. Anything that spawns creatures do not have to check if the creature they're spawning has an immediate effect or not. Unlike the alternative, this avoids needing code duplication across Hole, Vent, ContractedWorker etc. for doing this check, adhering to DRY. The Spawner concrete classes also shows high cohesion as they group the spawning logic and the immediate effects together, which is more readable and maintainable.

REQ5 (Yang Li Shen, 36031542)

Design 1: Emotion Based State Pattern

Transition logic lives in `game.states` (`ScavengeState`, `PanicState`, `LureState`, etc.), orchestrated by `StatefulCreature` and concrete actors `Muckraker` and `PhantasmWisp`. Each creature registers four State objects in a `TreeMap<Emotion, State>`. `playTurn` calls `transition`, updates `currentEmotion` when it changes, runs `immediateEffect`, then executes that state's behaviours. Each state class owns deterministic rules (worker count, adjacency, inventory fullness) and entry effects (steal valuable item, stun via `StunStatus`, push/pull ground items).

every creature has four distinct states, non-random transitions driven by surroundings, each state can reach multiple successors (e.g. `ScavengeState` to `Curious`, `Greedy`, or `Fearful`), and transitions fire immediate complex effects separate from turn behaviours. Hypothetical example: a `Muckraker` with a full inventory and no workers nearby leaves `Scavenge` and enters `Hoard`; on entry it drops a random inventory item while later turns wander. Maintainability: new moods add a class plus one `addNewState` line without rewriting other states. Scalability: `PhantasmWisp` reuses the same framework with different transition geometry (adjacent vs 2-4 tile bands). DRY centralises worker scanning in `State`; KISS keeps `StatefulCreature` a thin coordinator. SOLID: SRP splits each mood's rules; OCP extends by new `State` subclasses; Liskov substitutability via the shared `State` API. Connascence: of name links each `Emotion` enum constant to its `State` class (moderate; rename requires coordinated edits); of type binds every state to `StatefulCreature` through the constructor (strong; states cannot attach to arbitrary actors). Together, these tighten compile-time safety but couple states to this actor family. Extension: add `GluttonState` extends `State` for a fifth greedy variant on `Muckraker`; register it in the constructor (association). On transition to `Glutton`, `immediateEffect` could consume a ground item while `GrowBehaviour` handles turns; `StatefulCreature` still drives the pipeline.

Pros	Cons
Isolated state classes make transition maps easy to trace and test per mood.	Eight state classes increase navigation overhead for small changes.
Shared <code>State</code> helpers avoid duplicating worker detection across eight states.	Emotion-to-class mapping is scattered across two actor constructors.
Both creatures share one lifecycle without copy-pasting <code>playTurn</code> logic.	Strong type binding to <code>StatefulCreature</code> limits reusing states on non-stateful actors.

Design 1: Alternative Design

A monolithic actor would store `currentState` as an int or enum and implement all transitions and effects inside `Muckraker/PhantasmWisp` with large switch blocks and private helper methods. Hypothetical example: entering panic in one switch branch would stun neighbours inline instead of delegating to `PanicState.immediateEffect`. Maintainability degrades as each new mood lengthens the same file. Scalability across two creatures duplicates switch trees. DRY suffers unless helpers are extracted anyway; KISS looks simpler initially but hides growing complexity. SOLID: SRP violated when one class owns eight transition graphs; OCP closed; every new mood edits the actor. Connascence comparison: the alternative raises of position (transition order inside one method) and of execution (shared mutable fields touched by every branch), generally stronger than Design 1's of name enum map. Weaker type connascence per class, but overall coupling is higher because any edit risks unrelated states. Trade-off: fewer files vs fragile mega-classes. Extension: `MuckrakerStateHandler` as a separate class still associated with `Muckraker`; the actor delegates to it; partial recovery of SRP at the cost of an extra layer.

Pros	Cons
Fewer types to open when debugging a single creature.	Eight moods for two creatures invite duplicated predicates and copy-paste bugs.
All transition rules visible in one scrollable unit.	Mixing transition, effect, and behaviour logic weakens cohesion.
No abstract State hierarchy to learn.	Refactoring one mood risks regressions in unrelated branches.

Design 2: Prioritized Engine Behaviour Composition

Per-turn movement and item interaction are delegated to engine Behaviour<Actor, Action> implementations in game.behaviours, registered inside each State via addNewBehaviour(priority, ...). ScavengeState wires PickItemBehaviour and FindItemBehaviour; PanicState uses FleeBehaviour; LureState uses SwapItemsBehaviour; TinkerState uses TeleportBehaviour. StatefulCreature iterates behaviours in priority order after transitions - matching FIT2099's intended engine usage.

At least two states per creature expose new ground-level behaviours (pickup chain, flee, swap, teleport) with non-trivial logic, while transitions and immediateEffect remain in state classes. Hypothetical example: in Curious TinkerState, TeleportBehaviour relocates the nearest worker while entry fire on adjacent tiles runs separately in immediateEffect. Maintainability: behaviours are unit-testable without simulating full emotion graphs. Scalability: reuse FleeBehaviour on any actor by passing range. DRY shares movement algorithms; KISS keeps states as declarative wiring lists. SOLID: SRP separates "how to act" from "when to change mood"; ISP - states depend only on Behaviour's operate. Connascence: of meaning - priority integers must align with intended precedence (moderate); of type between Behaviour and Action (weak, engine-standard). Extension: add ConfuseBehaviour and register it in a new ChaosState at priority 1; ChaosState depends on ConfuseBehaviour (dependency), collaborates through stateBehaviourAction() like existing states.

Pros	Cons
New behaviours plug in without rewriting transition code.	Worker-search logic is duplicated between FleeBehaviour and State helpers.
Aligns with engine patterns markers expect for NPC turns.	Priority numbers are implicit contracts not enforced by the type system.
Behaviours can be recombined across creatures and future actors.	Debugging requires tracing both state transition and behaviour chain each turn.

Design 2: Alternative Design

Inline behaviour methods inside each State subclass (e.g. private Action tryPickup(Actor, Location)) would replace separate behaviour classes. Hypothetical example: ScavengeState would call an internal pickup helper instead of PickItemBehaviour. Maintainability collapses when two states need similar movement - copy-paste or fat base class. Scalability to a third creature reimplements flee logic again. DRY worse than Design 2 unless lifted late; KISS fewer packages. SOLID: SRP blurs state as mood controller and action generator. Connascence comparison: inline design shows stronger of execution (shared private fields in one state object) versus Design 2's weaker of type across small behaviour classes - Design 2 minimises coupling but adds files and indirection. Trade-off: lower class count vs harder reuse. Extension: extract ItemSwapLogic utility called from multiple states - hybrid that slowly recreates behaviour objects.

Pros	Cons
Single file per mood shows both transition and action code together.	Duplicated flee/pickup code across states increases defect risk.
No indirection through behaviour priority maps.	State classes grow until they violate SRP.
Slightly fewer classes in the diagram.	

Comparison of Alternative Designs

The monolithic-switch alternative concentrates connascence of position and execution in one actor type, so a small requirement tweak can ripple through unrelated moods. The inline-behaviour alternative reduces class count but reproduces connascence of algorithm across state subclasses whenever two moods share movement. Between alternatives, the monolithic handler delegate weakens execution connascence slightly yet retains centralised branching; inline methods fare worst for reuse. The behaviour-delegation alternative (not chosen) would mirror Design 2 but at the cost of more abstraction layers - best coupling profile, highest file overhead.

CONCLUSION

The diagram above shows the final design of REQ 5, which utilizes Design 1 (emotion-keyed State objects coordinated by StatefulCreature) together with Design 2 (prioritized engine Behaviour classes for turn actions). Design 1 was chosen because it cleanly separates deterministic transition graphs and immediate effects for Muckraker and PhantasmWisp without procedural sprawl. Design 2 was chosen to host novel behaviours (PickItemBehaviour, FleeBehaviour, SwapItemsBehaviour, TeleportBehaviour) while keeping transition predicates in state classes - balancing KISS in the turn loop with DRY reuse of engine interfaces. SOLID alignment (SRP, OCP, Liskov on State) supports the eight moods across two creatures within current scope. Connascence of name between Emotion and State is an acceptable trade-off: it is explicit, searchable, and weaker than mega-switch execution coupling, which aids long-term maintenance. Trade-offs include duplicated worker scanning between FleeBehaviour and State helpers and eight state source files to navigate. Monolithic or fully inline alternatives may suit larger teams or engines without behaviour lists, but would sacrifice modularity gained here.